

Explicação do Código – Análise de Segurança Pública de Saquarema

Introdução

Este documento tem como objetivo explicar, de forma detalhada, todo o código Python desenvolvido para a análise de segurança pública do município de Saquarema. A ideia do projeto surgiu dentro da disciplina de Big Data, onde a gente precisava pegar um conjunto de dados real (ou realista) e aplicar técnicas de análise, visualização e até inteligência artificial para extrair informações úteis. A gente escolheu trabalhar com dados de ocorrências criminais porque é um tema que impacta diretamente a vida das pessoas e que pode gerar insights importantes para políticas públicas.

O código foi escrito em Python, que é uma das linguagens mais usadas para ciência de dados hoje em dia. Ele faz tudo de forma automatizada: lê a planilha, limpa os dados, filtra o que interessa, treina um modelo de regressão linear e gera gráficos profissionais. Vamos explicar cada parte a seguir.

Bibliotecas Utilizadas

Logo no início do código, a gente importa quatro bibliotecas que são essenciais para o projeto funcionar:

- `pandas` (importado como `pd`): é a principal biblioteca para manipulação de dados em Python. Com ela a gente consegue ler o arquivo Excel, filtrar linhas, agrupar informações por mês ou bairro, e fazer praticamente qualquer operação com tabelas de dados. Sem o `pandas`, o projeto não existiria.
- `matplotlib.pyplot` (importado como `plt`): é a biblioteca de visualização. É com ela que a gente cria todos os gráficos — de linha, de barra, de pizza. Ela permite personalizar cores, títulos, legendas e salvar as imagens em alta resolução.
- `sklearn.linear_model.LinearRegression`: faz parte do `scikit-learn`, que é a biblioteca de machine learning mais popular do Python. A gente usa especificamente a classe `LinearRegression` para criar um modelo de regressão linear, que é uma técnica de inteligência artificial que tenta encontrar uma linha reta que melhor represente a tendência dos dados ao longo do tempo.

- numpy (importado como np): é uma biblioteca para cálculos numéricos. No nosso código, a gente usa ela principalmente para criar sequências numéricas (com np.arange) e para gerar cores nos gráficos (com np.linspace).

Etapa 1 – Leitura do Arquivo Excel

A primeira coisa que o código faz é ler o arquivo "seguranca_saquarema_bigdata.xlsx" usando o comando `pd.read_excel()`. Esse comando transforma a planilha inteira em um DataFrame do pandas, que é basicamente uma tabela dentro do Python onde cada coluna tem um nome e cada linha é um registro.

Logo depois de ler, a gente padroniza os nomes das colunas: remove espaços extras e converte tudo para letras minúsculas. Isso é importante porque às vezes o Excel tem espaços invisíveis ou letras maiúsculas que podem causar erros no restante do código. Por exemplo, se uma coluna se chama " Tipo " com espaços, o código não vai encontrar quando a gente tentar acessar `df["tipo"]`. Com a padronização, isso é resolvido automaticamente.

Também imprimimos na tela o total de registros e os nomes das colunas, para que quem estiver rodando o código saiba exatamente com o que está lidando.

Etapa 2 – Limpeza e Ajuste dos Dados

Essa é uma das etapas mais importantes em qualquer projeto de dados. Dados do mundo real quase sempre vêm bagunçados, com valores faltando, nomes escritos de formas diferentes, datas em formatos estranhos e por aí vai. Se a gente não limpar antes de analisar, os resultados podem sair completamente errados.

Primeiro, o código procura automaticamente qual coluna contém a quantidade de ocorrências. Ele faz isso verificando se o nome da coluna contém a palavra "quant" ou "ocorrencias". Se encontrar, renomeia para "quantidade". O mesmo acontece com a coluna de tipo de crime — ele procura qualquer coluna que contenha "tipo" no nome e renomeia para "tipo". Essa abordagem é inteligente porque funciona mesmo que a planilha original tenha nomes ligeiramente diferentes, como "Quantidade_Total" ou "tipo_crime".

Depois, a coluna "tipo" é convertida para texto, removem-se espaços e tudo vira minúsculas. Isso garante que "Roubo", "roubo", "ROUBO" e " roubo " sejam todos tratados como a mesma coisa.

A coluna "data" é convertida para o formato de data do Python usando `pd.to_datetime()`. O parâmetro `errors="coerce"` faz com que valores que não são datas válidas sejam transformados em NaN (valor nulo) em vez de causar um erro. Depois, o `dropna()` remove todas as linhas que não têm data ou quantidade válida.

Por último, duas novas colunas são criadas a partir da data: "mes" (que agrupa por mês/ano, tipo 2023-01, 2023-02) e "ano" (que extrai só o ano). Essas colunas são usadas depois para fazer os agrupamentos nos gráficos.

Etapa 3 – Filtro de Roubos

Aqui a gente cria um subconjunto dos dados focando apenas nos registros de roubo. O código usa `df["tipo"].str.contains("roubo")` para selecionar todas as linhas onde o tipo de crime contém a palavra "roubo". Isso pega tanto "roubo" quanto "roubo de veículo", "roubo a transeunte" e qualquer outra variação.

Depois, esses dados de roubo são agrupados por mês usando o `groupby("mes")`, e a quantidade total de ocorrências de cada mês é somada. O resultado é uma tabela simples com duas colunas: o mês e o total de roubos naquele mês. Essa tabela é a base para o gráfico de evolução e para o modelo de IA.

Também é criada uma coluna chamada "tempo", que é simplesmente uma sequência numérica (0, 1, 2, 3...) representando a ordem dos meses. Essa coluna é necessária para a regressão linear, porque o modelo precisa de números, não de datas.

Etapa 4 – Inteligência Artificial (Regressão Linear)

Essa é a parte mais interessante do código e onde entra a inteligência artificial de verdade. A gente usa um algoritmo chamado Regressão Linear, que é um dos modelos mais clássicos de machine learning.

A ideia da regressão linear é simples: dado um conjunto de pontos (no nosso caso, os totais mensais de roubo ao longo do tempo), o algoritmo tenta traçar a melhor reta possível que passa por esses pontos. Se a reta vai para cima, significa que os roubos estão aumentando com o tempo. Se vai para baixo, estão diminuindo. Se fica mais ou menos plana, significa que está estável.

No código, a variável X (maiúscula, como é convenção em machine learning) recebe a coluna "tempo" e a variável y recebe a coluna "quantidade". O modelo é criado com `LinearRegression()` e treinado com `modelo.fit(X, y)`. Depois de treinado, ele gera previsões com `modelo.predict(X)`, que são os valores que a reta passa em cada ponto no tempo.

O R^2 (R-quadrado) é a métrica que mede a qualidade do modelo. Ele varia de 0 a 1:

- $R^2 \geq 0.7$: modelo bom — a tendência linear explica bem o comportamento dos dados.
- R^2 entre 0.4 e 0.7: modelo moderado — existe uma tendência, mas com bastante variação.
- $R^2 < 0.4$: modelo fraco — os dados variam muito e a linha reta não consegue capturar o padrão.

O código imprime automaticamente a interpretação do R^2 na tela, para que qualquer pessoa consiga entender se o modelo foi bom ou não sem precisar de conhecimento técnico.

Etapa 5 – Geração dos Gráficos

Essa é a parte visual do projeto. A gente gera quatro gráficos diferentes, cada um mostrando uma perspectiva diferente dos dados. Antes de criar os gráficos, tem um bloco de configuração visual (`plt.rcParams.update`) que define o fundo claro, ativa as grades e ajusta o tamanho da fonte para todos os gráficos ficarem com uma aparência profissional e consistente.

Gráfico 1 – Evolução de Roubos com Tendência de IA

Este é o gráfico principal do projeto. Ele mostra um gráfico de linha com os dados reais de roubos por mês (em azul, com marcadores nos pontos) e a linha de tendência gerada pela regressão linear (em vermelho tracejado). A área entre as duas linhas é preenchida com um tom avermelhado transparente, o que ajuda a visualizar as diferenças entre o valor real e o previsto pelo modelo.

No título do gráfico aparece o valor do R^2 , para que quem olhar já saiba a qualidade do modelo. Os meses ficam no eixo X (rotacionados para não sobrepor) e o número de ocorrências no eixo Y. O gráfico é salvo como "grafico1_evolucao_roubos.png" com 150 dpi de resolução.

Gráfico 2 – Top 5 Bairros com Mais Roubos

Este gráfico é um gráfico de barras horizontais que mostra quais são os 5 bairros de Saquarema com mais ocorrências de roubo. Os dados são agrupados por bairro usando `groupby("bairro")` e depois selecionados os 5 maiores com `nlargest(5)`. As barras usam uma paleta de tons de azul, indo do mais escuro (primeiro lugar) ao mais claro (quinto lugar). Ao lado de cada barra aparece o valor exato de ocorrências, para facilitar a leitura.

Esse gráfico é muito útil para identificar as áreas mais críticas da cidade e direcionar policiamento ou outras medidas de segurança.

Gráfico 3 – Distribuição de Todos os Crimes

Diferente dos gráficos anteriores que focam só em roubo, este é um gráfico de pizza (ou gráfico de setores) que mostra a proporção de TODOS os tipos de crime registrados na planilha. Cada fatia representa um tipo de crime, e a porcentagem aparece dentro de cada fatia.

Os nomes dos crimes são formatados para ficarem mais legíveis: underlines são substituídos por espaços e cada palavra começa com maiúscula (usando `.title()`). A paleta de cores usada é a Set3 do matplotlib, que tem cores bem distintas entre si, facilitando a diferenciação visual entre os tipos de crime.

Esse gráfico dá uma visão geral da criminalidade: será que roubo é o crime mais comum, ou será que furto ou outro tipo domina? A pizza responde isso de forma bem direta.

Gráfico 4 – Tempo de Resposta por Bairro

Este gráfico só é gerado se a planilha contiver uma coluna chamada "tempo_resposta_minutos". Se existir, ele calcula a média de tempo de resposta para cada bairro e plota um gráfico de barras horizontais.

O diferencial desse gráfico é a coloração condicional: bairros onde o tempo de resposta está acima da média geral aparecem em vermelho, e os que estão abaixo (ou seja, mais rápidos) aparecem em verde. Uma linha vertical tracejada preta marca exatamente onde está a média geral, com o valor em minutos na legenda.

Esse é um gráfico com muita utilidade prática: ele mostra quais bairros estão sendo mal atendidos em termos de velocidade de resposta policial, o que pode indicar falta de viaturas, distância geográfica dos batalhões ou outros problemas operacionais.

Etapa 6 – Resumo Final

No final do código, é impresso um resumo organizado na tela com as principais informações extraídas da análise. Ele mostra:

- O total de registros que foram analisados.
- O período coberto pelos dados (de qual ano a qual ano).
- Qual é o bairro com mais roubos.
- Qual é o tipo de crime mais frequente em toda a base.

- O tempo médio de resposta (se a coluna existir na planilha).
- A precisão do modelo de IA (valor do R^2).

Esse resumo é importante porque nem todo mundo vai ler os gráficos ou entender o código. Com ele, qualquer pessoa consegue ter uma visão rápida dos principais achados em formato texto.

Considerações Finais

No geral, esse código mostra como é possível usar programação e ciência de dados para transformar uma simples planilha de ocorrências em análises visuais e inteligentes. A combinação de pandas para manipulação dos dados, matplotlib para visualização e scikit-learn para machine learning forma um pipeline completo que vai desde a leitura do dado bruto até a geração de insights acionáveis.

Uma coisa legal do código é que ele é genérico o suficiente para funcionar com planilhas de outros municípios também — bastaria trocar o arquivo de entrada. As renomeações automáticas de colunas e a detecção condicional de colunas (como a de tempo de resposta) tornam o código flexível e reutilizável.

O projeto demonstra na prática o que a gente aprende em Big Data: coletar, limpar, processar, modelar e visualizar dados para gerar conhecimento. E o mais importante: tudo isso pode ser usado de verdade para melhorar a segurança pública de uma cidade como Saquarema.